No.

4-dars: Advanced Feature Selection & Dimensionality Reduction

- Eng muhim featurelarni tanlab olish model yaxshi ishlashini / yaxshi predict qilishni taminlaydi. → Ya'ni mantiqsiz javob qilmaydi

Teskarisi yaxshi ishlashi - overfitting bolishi mumkin sekin ishlashi yoki Bias bolishi m-k.

! Asosiysi: Biz iloji boricha Accuracy score baland chiqishda emas, model stabil ishlashi, togri predict qilish haqida ham qayqurishimiz k-k.

Shuning uchun ham biz Featurelarni filter qilishimiz k-k.

Ya'ni har doim ham bizning qolimizda kerakli yaroqli featurelar bolmaydi. Hamma materiallarni shunday tashlab qoyishadi. Biz ularni soxa bilmidan kelib chiqib olishimiz kerak boladi.

- Ovqat == Fazat kerakli masalliqlar.

Nima uchun foydali?:

I. Reduces overfitting
II. Speeds up training
III. Makes models easier to interpret (Osonroq tushunishimiz)
IV. Improves generalization (
V. Helps remove noisy or redundant feature
(Keraksiz va mantiqsiz bolgan featurelarni tashlab yubaришimizga kerak boladi)

- *** Kop masalliq ovqat taminni buzib qoyadi.

No.

Method:

- Uy qurishdan oldin:
  - Qaysi materiallar kerakli
  - Qaysilari eng muhim

! Method 1: Correlation Filtering + Mutual Information
(Preprocesing va FE bilishi kerak)
Ya'ni o'zaro bog'liqligiga ko'ra featurelarni tanlab oladi.
Ya'ni Input qiymatini Output qiymatlarga bog'liqligi jihatidan tanlab oladi.

! Method 2: ANOVA F-test (Categorical → Continuous target)
Continuous qiymatlari ~~bo'ch~~ bo'yicha Categorical qiymatlarni aniqlaydi.

! Method 3: Tree Based Feature Selection
Complex xolatlarda yaxshi natija beradi.

Method 1: → Amaliy

```
from sklearn.feature_selection import mutual_info_classif
import pandas as pd

corr = df.corr()['churn'].sort_values(ascending=True)

X = df.drop('Churn', axis=1)
y = df['Churn']
```

No.

```
mi = mutual_info_classif(X, y, random_state=42)
mi_series = pd.Series(mi, index=X.columns).
                                sort_values()

print("Correlation Ranking:\n", corr)
print("\n Mutual Information Ranking:\n",
                                mi_series)
```

Method 2: → Amaliy

```
from sklearn.feature_selection import f_classif
import pandas as pd
```

qancha yu qori bolsa shuncha yaxshi

→ Qancha past bolsa shuncha yaxshi

```
f_values, p_values = f_classif(X, y)

f_df = pd.DataFrame({
        'Feature': X.columns,
        'F_value': f_values,
        'P_value': p_values

f_df = f_df.sort_values('F_value',
                        ascending=False)

print(f_df)
```

=> Buning natijalini % emas statistik qiymat deb olish k-k!

F - value - kattasi muhim
p - value - kichigi muhim

```
top_k = 10    # threshold qiymat
selected_features = f_df.head(top_k)['Feature'].tolist()
print(selected_features)
```

Savol bo'ildi: Contract feature Method 1 da (−) edi ammo Method 2 da (+) nega?
Sababi agar 2si ham bir xil natija bersa unda bu Methodlar yaratilishidan mano nima? Method 1 boshqa rakursdan, Method 2 esa boshqa rakursdan qaraydi. → Sizning datasetingizga qaysi biri yaxshiroq ta'sir qilsa deylik Correlation bilan yuqorida belgilab olingan (+) qiymat/ni oldik ~~deylik~~. Va endi ANOVA bilan topasiz va k-si ni ham tabu lab ga qo'yasiz → Qaysi biri yaxshi chiqsa o'sha asal siz u-n eng yaxshisi bo'ladi !!!

Method 4: L1 (Lasso) Regularization # linear modellar

Bu usulimiz linear Modellar uchun ishlaydigan usulimiz hisoblanadi. Yani Linear modellar bo'lsa sizning modelingizda (linear modellarda) featurelarni tanlab olmoqchi bo'lsangiz. Shu L1 Lassodan foydalanasiz. Birdaniga model qilib o'zi topib ketadi.

No.

```
lasso_selected = lasso_df[lasso_df['Importance']>0]
                 ['Feature'].tolist()
print("\n Selected Features (Non-zero Coefficients):")
print(lasso_selected)
```

Natijada $\boxed{\text{lasso_selected}}$ dan chiqqan listni olib

```
X = [ .... ]
y = d['churn']
```

} deya fit, predict qilib tekshirib ko'radi!

Method 3 - Non Linear (Tree Based Feature Importance)

=> Tree-Based Algoritm: ~~zori~~ zor domoni o'zlariz ichida feature importance'i bor. O'zini ichida muhim bo'lgan featurelarni aniqlay oladi.

```
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

rf = RandomForestClassifier(n_estimators=200,
                            random_state=42)
rf.fit(X, y)

rf_importances = rf.feature_importances_
rf_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_importances,
}).sort_values('Importance', ascending=False)
print("===RANDOM FOREST IMPORTANCE (high→low)===")
print(rf_df)
```

Eng muhim qismi

```
rf_threshold = 0.02
Nrf_selected = rf_df[rf_df['Importance'] > rf_threshold]
['Feature'].tolist() & top10_rf = rf_df.head(10)['Feature'].tolist()
print("\nTop 10 Tree-Based Features")
print(top10_rf)
```

Dimensionality Reduction

- Multicollinearity === Bir-biriga juda yaqin bolgan feature lar
- 2 xil yutuch bor faqat bir xilidan osh qilamiz.
- PCA === eng muhim bolgan featurelarni aniqlamaydi
- PCA === 2 ta bir biriga yaqin bolgan featurelardan ularni ozni bosadigan yangi feature yaratadi.
- Masalan: Boy va vazndan yangi tana-olchami degan
- Original features → best combination → Yangi feature → drop original => Shu workflowda ishlayd
- 3 xil mevadan alohida-alohida ishlatgandan kora ularni blendr qilib sharbat qilib ichgan maqul.

Nima uchun kerak?:
- Dimensionality reduction
- Noise reduction
- Better model performance
- Faster computation

Unsupervised ML ning muhim bolgan Data Preprocessing bolimi Standard Scaler hisoblanadi.

No.

PCA -> Principal Component Analysis)

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import pandas as pd
import numpy as np

X = df.drop('Churn', axis=1)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA()
X_pca = pca.fit_transform(X_scaled)

explained_var = pca.explained_variance_ratio_

pca_df = pd.DataFrame({
    'Principal Component': [f'PC{i+1}' for i in range
                            (len(explained_var))],
    'Explained Variance (%)': explained_var * 100,

    'Cumulative Variance (%)': np.cumsum(explained_var)
                               * 100
})
print("=== PCA Explained Variance Table ===")
print(pca_df)
```

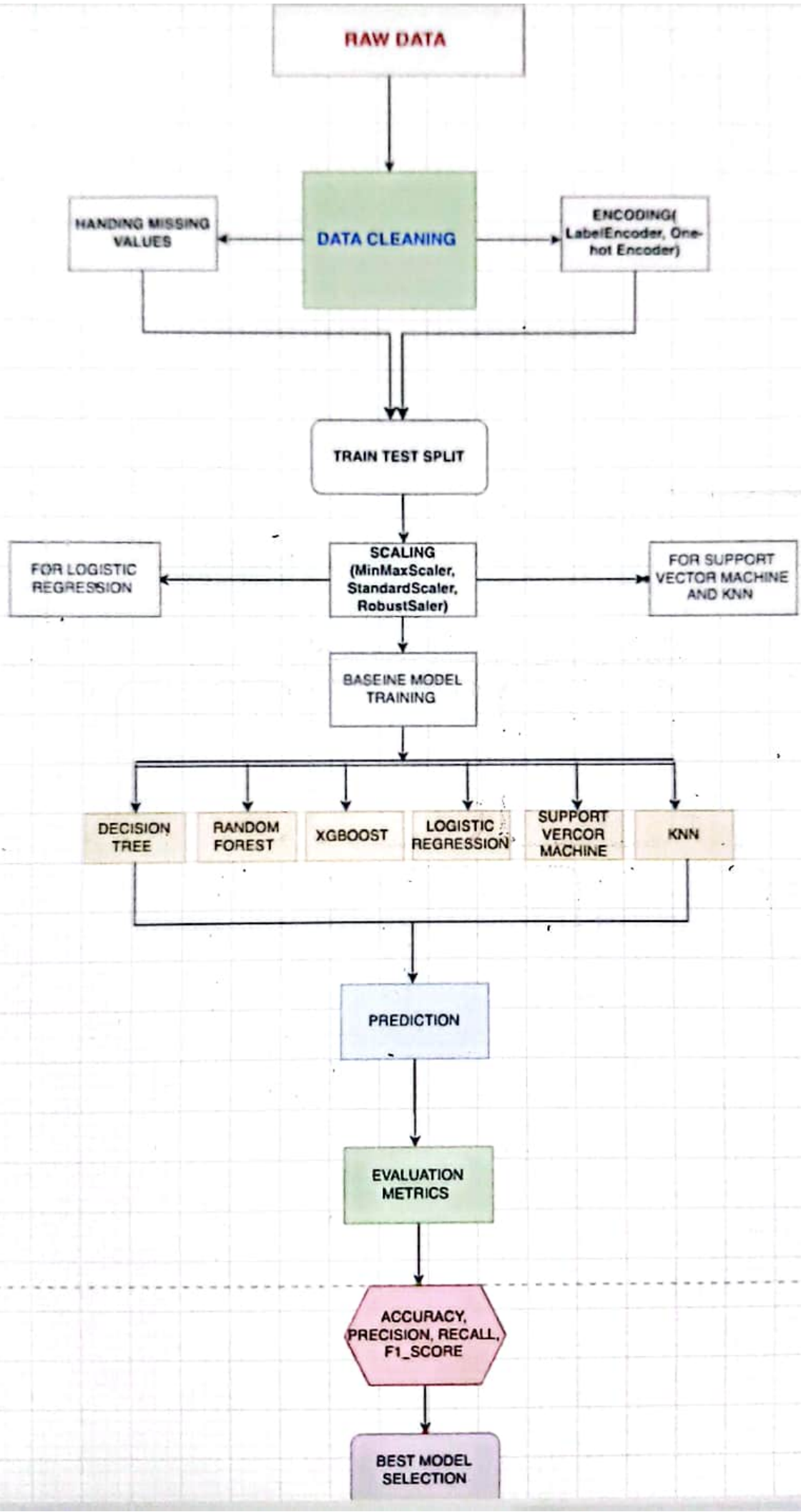
No.

```
n_components = (np.cumsum(explained_var) <
                0.90).sum() +1
print(f"\n Recomended number of components:
        {n_components}")
```

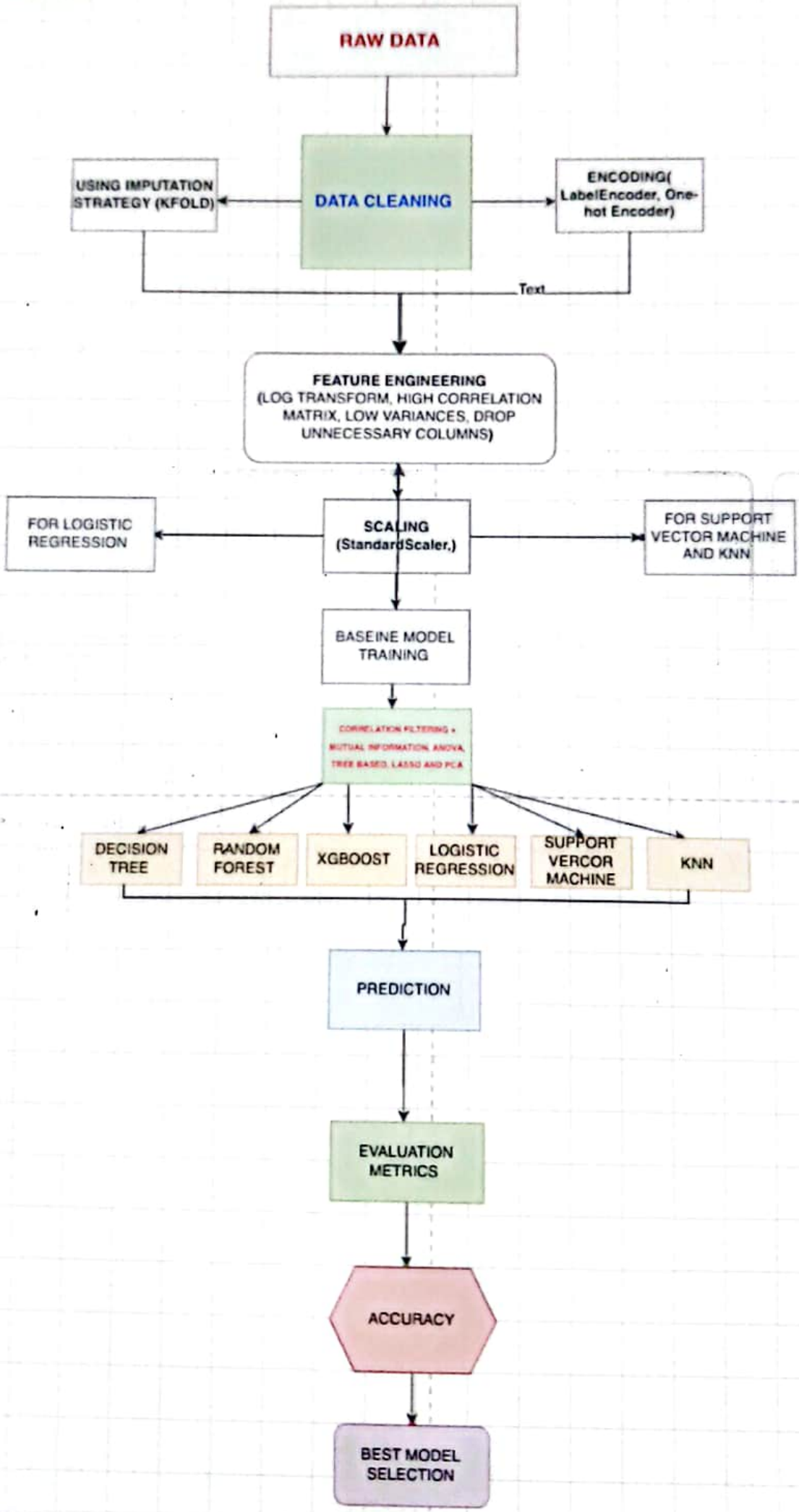
#5. Reduce data using selected components

```
pca_final = PCA(n_components = n_components)
X_reduced = pca_final.~~fit~~fit_transform(X_scaled)

print("\n shape before PCA:", X.shape)
print("Shape after PCA:", X_reduced.shape)
```


RAW DATA
DATA CLEANING
HANDING MISSING VALUES
ENCODING(LabelEncoder, One-hot Encoder)
TRAIN TEST SPLIT
SCALING (MinMaxScaler, StandardScaler, RobustSaler)
FOR LOGISTIC REGRESSION
FOR SUPPORT VECTOR MACHINE AND KNN
BASEINE MODEL TRAINING
DECISION TREE
RANDOM FOREST
XGBOOST
LOGISTIC REGRESSION
SUPPORT VERCOR MACHINE
KNN
PREDICTION
EVALUATION METRICS
ACCURACY, PRECISION, RECALL, F1_SCORE
BEST MODEL SELECTION

CamScanner


RAW DATA
DATA CLEANING
USING IMPUTATION STRATEGY (KFOLD)
ENCODING(LabelEncoder, One-hot Encoder)
Text
FEATURE ENGINEERING (LOG TRANSFORM, HIGH CORRELATION MATRIX, LOW VARIANCES, DROP UNNECESSARY COLUMNS)
SCALING (StandardScaler,)
FOR LOGISTIC REGRESSION
FOR SUPPORT VECTOR MACHINE AND KNN
BASEINE MODEL TRAINING
CORRELATION FILTERING + MUTUAL INFORMATION, ANOVA, TREE BASED, LASSO AND PCA
DECISION TREE
RANDOM FOREST
XGBOOST
LOGISTIC REGRESSION
SUPPORT VERCOR MACHINE
KNN
PREDICTION
EVALUATION METRICS
ACCURACY
BEST MODEL SELECTION

CamScanner